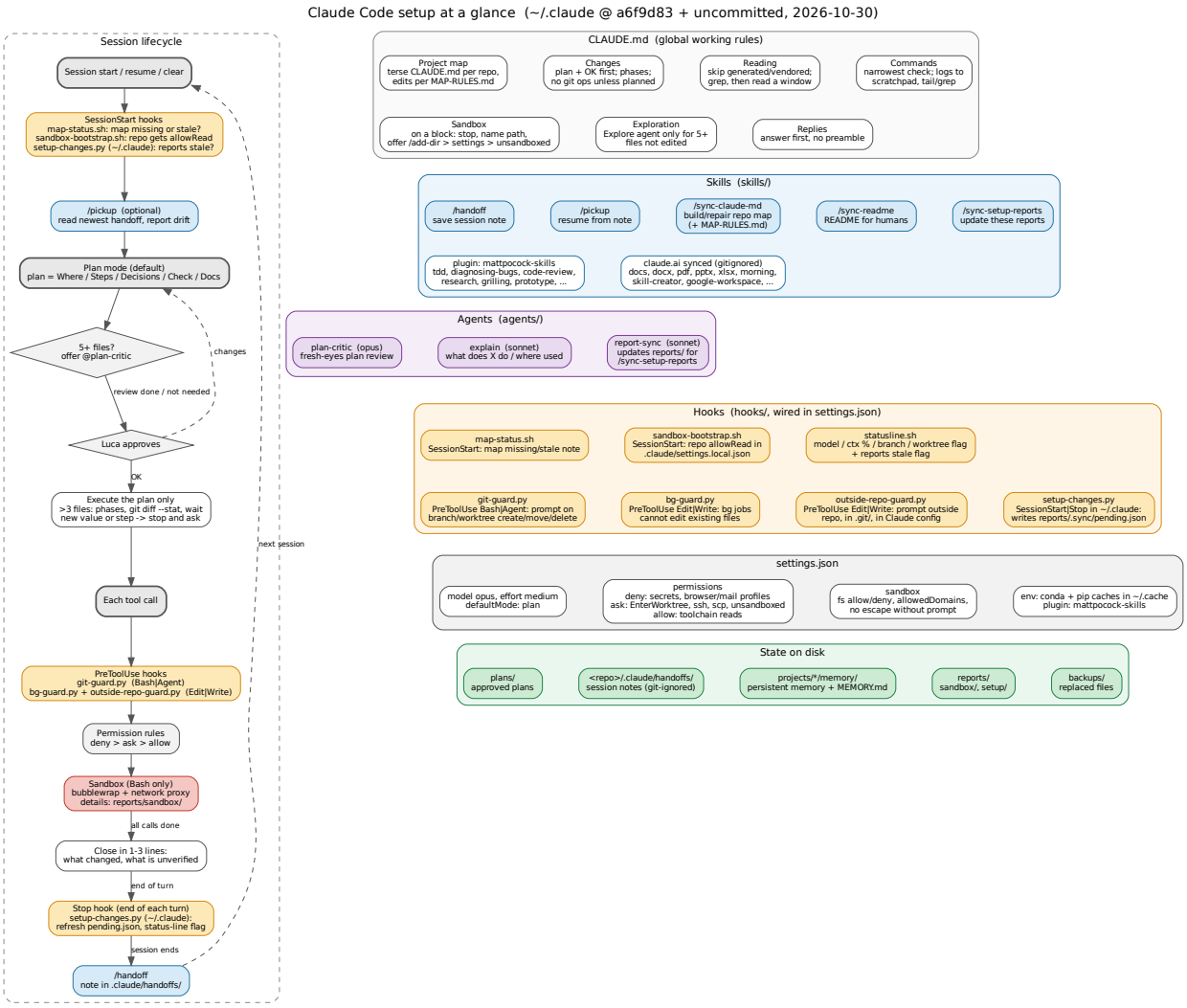
Claude Code setup: overview

`~/.claude` @ `a6f9d83` + uncommitted

2026-10-30

Goal. Claude works the way a careful collaborator would: it plans and waits for approval before any change, never creates branches or worktrees by surprise, keeps a terse map of each repo current, and leaves handoff notes so a fresh session can resume. This report shows the pieces that make that happen; the sandbox has its own report in `../sandbox/sandbox-structure.pdf`.

Enforcement legend. HARNESS enforced by Claude Code itself HOOK enforced by one of the hooks below RULE instruction in `CLAUDE.md` only (Claude follows it; nothing checks).



1 Working rules (CLAUDE.md)

Global instructions loaded in every session.

Section	Rule	Enforced by
Project map	Each repo has a terse `CLAUDE.md` map; trust it, flag it when stale, edit it by `skills/sync-claude-md/MAP-RULES.md`. Never applied in `~/.claude` itself.	HOOK `map-status.sh` + RULE
Changes	Plan first (Where / Steps / Decisions / Check / Docs) and wait for OK; every chosen value listed; >3 files in phases; 5+ files: offer `@plan-critic`; no branch, commit, push, stash unless planned.	HARNESS plan mode + HOOK `git-guard.py` + RULE
Reading	Skip generated and vendored files; in large files `grep -n`, then read a window.	RULE
Commands	Narrowest check first, full suite once; noisy output to a log, then `tail`/`grep`; quiet modes.	RULE
Sandbox	On a block, stop and name the path or host; offer `/add-dir`, then a settings edit, then an unsandboxed retry. Never work around it.	HARNESS sandbox + HOOK `outside-repo-guard.py`
Exploration	Explore subagent only when 5+ files would be read and not edited.	RULE
Replies	Answer first, no preamble; after changes close in 1–3 lines.	RULE

2 Skills

Invoked as `/name`. Own skills have `disable-model-invocation`: they run only when typed.

Skill	Source	What it does
`/handoff [focus]`	own	Saves the session (goal, state, decisions, open questions, next step, pointers) as a note in `<repo>/.claude/handoffs/`.
`/pickup [slug]`	own	Reads a handoff and its pointers, compares with git now (branch, commits, uncommitted files), reports in ≤5 lines, waits.
`/sync-claude-md [path]`	own	Builds or repairs a repo's `CLAUDE.md` map: draft, OK, write. Rules in `MAP-RULES.md`. Runs in the main session (a fork would inherit plan mode).
`/sync-readme [path]`	own	Builds or refreshes a `README.md` for human readers, with research sections and a fork rule: draft, OK, write.
`/sync-setup-reports`	own	Updates these reports after setup changes: lists the changed files, runs the `report-sync` agent, shows the diff for review, then stamps the snapshot on OK.
mattpocock-skills	plugin	`tdd`, `diagnosing-bugs`, `code-review`, `research`, `grilling`, `prototype`, `domain-modeling`, `codebase-design`, `pr`, `wizard`, `writing-for-agents`.
claude.ai synced	account	`docs`, `docx`, `pdf`, `pptx`, `xlsx`, `morning`, `skill-creator`, `google-workspace`, `import-memory`, … In `skills/synced/`, git-ignored, refreshed by sync.

3 Agents

Subagents in `agents/`. `explain` and `plan-critic` are read-only (Read, Grep, Glob): they can look, never change.

Agent	Model	When it is used
`explain`	sonnet	"What does X do / how does X work / where is X used" about code not yet read in the session. Cheaper model; the answer comes back as a summary.
`plan-critic`	opus	Fresh-eyes review of a plan before approval: on request, or offered for plans touching 5+ files.
`report-sync`	sonnet	Used only by `/sync-setup-reports`: updates `reports/` from `reports/.sync/pending.json` and rebuilds them. Tools: Read, Grep, Glob, Edit, Write, Bash.

4 Hooks

Scripts in `hooks/`, wired in `settings.json`. A PreToolUse hook can let a call through, force a prompt, or deny it.

Event (matcher)	Script	Effect
SessionStart (startup, clear)	`map-status.sh`	Tells Claude when the repo has no map, or when ≥3 source files or any manifest changed since the map's verified commit.
SessionStart (startup, resume, clear)	`sandbox-bootstrap.sh`	Adds the repo to `allowRead` in its `.claude/settings.local.json` (the sandbox denies `~/` by default) and keeps that file out of git.
SessionStart (startup, resume, clear), Stop	`setup-changes.py`	In `~/.claude` only: compares the setup files with the snapshot of the last report sync and writes the difference to `reports/.sync/pending.json`.
PreToolUse (Bash, Agent)	`git-guard.py`	Forces a prompt before creating, moving or deleting a branch or worktree, and for Agent `isolation:worktree`.
PreToolUse (Edit, Write, NotebookEdit)	`bg-guard.py`	In a background job (`$CLAUDE_JOB_DIR` set): denies edits to existing files outside the job dir, temp dirs and memory. New files stay allowed.
PreToolUse (Edit, Write, NotebookEdit)	`outside-repo-guard.py`	Forces a prompt for edits outside the repo, inside `.git/`, or to Claude's own hooks, settings, skills, agents, `CLAUDE.md`.
statusLine	`statusline.sh`	Model · context used · git branch, flagged inside a linked worktree, and in `~/.claude` when the reports are stale (`pending.json` exists).

5 Settings (`settings.json`)

Key	Value
Model	`opus`, effort `medium`.
`defaultMode`	`plan`: every session starts read-only until a plan is approved.
Permissions	**deny** Read/Edit of secrets (`~/.ssh`, `~/.gnupg`, keyrings, `pwd.txt`, credentials) and browser/mail profiles and caches; **ask** `EnterWorktree`, `ssh`, `scp`, unsandboxed Bash; **allow** reads of toolchain dirs (conda, caches, Jupyter, Mathematica, MATLAB, TeX, Rust, ...).
Sandbox	Bubblewrap around Bash, network allow-list, no escape without a prompt. Full rules: `../sandbox/sandbox-structure.pdf`.
Env	`CONDA_PKGS_DIRS`, `PIP_CACHE_DIR` in `~/.cache` (writable inside the sandbox).
Plugins	`mattpocock-skills`.
Theme	`dark-daltonized`.

6 State and files

Where	What
`plans/`	Approved plans, one file per plan (tracked).
`<repo>/.claude/handoffs/`	Handoff notes from `/handoff`, read by `/pickup`; git-ignored globally.
`projects/*/memory/`	Persistent memory per project, indexed by `MEMORY.md`.
`reports/`	This report (`setup/`) and the sandbox report (`sandbox/`). `reports/.sync/` (git-ignored): snapshot of the setup at the last sync and `pending.json`.
`backups/`	Copies of files replaced during setup.
`.gitignore`	Whitelist: tracks only `CLAUDE.md`, `settings.json`, `agents/`, `hooks/`, `skills/` (minus `synced/`), `plans/`, `reports/`, `.claude/` (minus `settings.local.json`).
`~/.config/git/ignore`	Global: `.claude/worktrees/`, `.claude/handoffs/`, `.claude/settings.local.json`, `CLAUDE.local.md`.

Rebuild (in `reports/setup/`):

```
dot -Tpdf setup-flow.dot -o setup-flow.pdf; dot -Tsvg setup-flow.dot -o setup-flow.svg
latexmk -pdf setup-overview.tex
```